Abstract

Error-correction learning is one of the fundamental supervised learning mechanisms used in artificial neural networks, particularly in single-layer models such as the perceptron. This report presents a practical demonstration of error-correction learning by training a perceptron model on two classical binary classification problems: the AND and OR logical functions. These tasks are linearly separable and serve as ideal introductory examples for analyzing learning behavior, convergence properties, and the impact of hyperparameters such as the learning rate.

The perceptron learning rule adjusts synaptic weights iteratively based on the difference between the predicted output and the desired target output. During training, the model computes the weighted sum of inputs, applies a step activation function, and updates weights whenever a misclassification occurs. This weight modification follows the error-correction principle, where changes are proportional to the learning rate, input values, and classification error. Over successive epochs, the perceptron progressively reduces classification errors until it finds a decision boundary that correctly separates the input patterns.

In this study, the perceptron algorithm originally introduced by Perceptron is applied to both AND and OR datasets. The decrease in classification error is plotted across training iterations to visualize the learning dynamics. The results demonstrate that for linearly separable problems such as AND and OR, the perceptron converges to a stable solution within a finite number of steps, consistent with the perceptron convergence theorem.

A key focus of this report is the analysis of learning rate effects on convergence speed and stability. Different learning rate values are tested to observe their influence on the error reduction curve. Smaller learning rates produce gradual and stable convergence but require more training epochs, whereas larger learning rates accelerate convergence but may cause oscillations before stabilization. The comparative analysis highlights the trade-off between convergence speed and stability in error-correction learning.

Overall, this demonstration provides a clear understanding of supervised learning dynamics in simple neural models. By examining error decrease plots and learning rate variations, the study reinforces theoretical concepts with empirical observation, illustrating how weight adaptation drives successful classification. The findings emphasize the importance of proper parameter selection and offer foundational insight into more advanced neural network training methods such as backpropagation.

1. Introduction

Error-correction learning is one of the fundamental supervised learning mechanisms used in Artificial Neural Networks (ANNs). It is based on the principle of minimizing the difference between the predicted output and the desired (target) output by adjusting the connection weights. This learning strategy forms the foundation of many modern training algorithms and plays a critical role in understanding how simple neural models learn logical and classification tasks.

One of the earliest and most important models trained using error-correction learning is the **Perceptron**, introduced by **Frank Rosenblatt** in 1958. The perceptron is a single-layer neural network used for binary classification problems. It computes a weighted sum of input features and applies a threshold activation function to produce an output of either 0 or 1 (or −1 and +1). The perceptron learning rule updates weights only when a misclassification occurs, making it a clear demonstration of error-driven adaptation.

Logical functions such as AND and OR are commonly used benchmark problems to illustrate perceptron learning because they are linearly separable classification tasks. A linearly separable problem is one where a single straight line (in two dimensions) or hyperplane (in higher dimensions) can correctly divide the input data into two classes. Training a perceptron on AND and OR tasks helps demonstrate how error-correction learning adjusts weights iteratively until the decision boundary correctly separates the classes.

2. Objectives

The primary aim of this experiment is to demonstrate how the error-correction learning rule enables a single-layer perceptron to learn linearly separable Boolean functions such as AND and OR. The specific objectives are:

1. To implement a Perceptron model for binary classification using simple logical datasets (AND and OR gates).
2. To apply the error-correction learning rule to update the weights iteratively based on classification error.

3. To observe the decrease in classification error across training epochs and visualize it using an error-versus-iteration plot.
4. To analyze the effect of learning rate (η) on:
    - Speed of convergence
    - Stability of training
    - Oscillatory or smooth weight updates
5. To verify convergence behavior for linearly separable problems and relate the results to theoretical guarantees of perceptron learning.
6. To understand decision boundary formation for AND and OR logic functions.

This experiment serves as a foundational demonstration of supervised learning and provides insight into how neural networks adjust internal parameters to minimize error.

3. Theoretical Background

2.1 The Perceptron Model

The perceptron is one of the earliest neural network models, introduced by **Frank Rosenblatt** in 1958. It is a single-layer feedforward neural network used for binary classification.

For an input vector:

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]$$

The perceptron computes a weighted sum:

$$v = \sum_{i=1}^{n} w_i \, x_i + b$$

where:

- w_i= weights
- b= bias
- v= net input

The output is obtained using a step activation function:

$$y = \begin{cases} 1, & \text{if } v \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

The perceptron forms a linear decision boundary, which is a straight line (in 2D) separating two classes.

2.2 Error-Correction Learning Rule

Error-correction learning is a supervised learning mechanism in which weights are adjusted proportionally to the error between desired and actual outputs.

The weight update rule is:

$$w_i^{new} = w_i^{old} + \eta(t - y)x_i$$

Bias update:

$$b^{new} = b^{old} + \eta(t - y)$$

where:

- η= learning rate
- t= target output
- y= predicted output
- $(t-y)$= error

This rule modifies the weights only when a misclassification occurs. If the prediction is correct, no update is made.

2.3 AND and OR Tasks

The AND and OR logic gates are classic examples of **linearly separable problems**.

AND Function

x1	x2	Target (t)
0	0	0
0	1	0
1	0	0
1	1	1

Only one positive output → decision boundary isolates (1,1).

OR Function

x1	x2	Target (t)
0	0	0
0	1	1
1	0	1
1	1	1

Only one negative output → decision boundary isolates (0,0).

Since both tasks are linearly separable, the perceptron is guaranteed to converge.

2.4 Error Decrease During Training

During training:

- Initially, weights are random.
- Misclassifications occur frequently.
- With each update, weights adjust toward correct classification.
- Total error per epoch gradually decreases.
- Eventually, the error becomes zero.

The error curve typically shows:

- A decreasing staircase-like pattern
- Rapid drop for suitable learning rate
- Oscillations for large learning rate

Plotting **epoch vs total error** visually demonstrates convergence.

2.5 Effect of Learning Rate (η)

The learning rate controls the magnitude of weight updates.

Small Learning Rate (η → small)

- Slow convergence
- Smooth error curve
- Stable training

Moderate Learning Rate

- Faster convergence
- Efficient training

Large Learning Rate

- Possible oscillations
- Instability
- May overshoot optimal weights

Thus, choosing an appropriate learning rate is crucial for stable and efficient learning.

2.6 Convergence Theory

The Perceptron Convergence Theorem states:

If a dataset is linearly separable, the perceptron learning rule will converge to a solution in a finite number of updates.

Since AND and OR are linearly separable:

- The perceptron always finds a separating hyperplane.
- Final error becomes zero.
- Weights stabilize.

4. Data Description

The Error-Correction Learning Demo uses simple logical gate datasets (AND and OR) to demonstrate how a perceptron learns through supervised learning. These datasets are chosen because they are linearly separable and suitable for training a single-layer perceptron.

1. Input Variables

Each dataset consists of two binary input variables:

- $x_1 \in \{0,1\}$
- $x_2 \in \{0,1\}$

A bias input $x_0 = 1$is also included to allow the model to adjust the decision boundary.

Thus, each training sample is represented as:

$$X = [1, x_1, x_2]$$

2. Output (Target) Values

The target output depends on the logical function being modeled.

AND Gate Dataset

x1	x2	Target (t)
0	0	0
0	1	0
1	0	0
1	1	1

The AND gate outputs 1 only when both inputs are 1.

OR Gate Dataset

x1	x2	Target (t)
0	0	0
0	1	1
1	0	1
1	1	1

The OR gate outputs 1 when at least one input is 1.

3. Dataset Characteristics

- Total samples: 4 for each task
- Type: Supervised learning dataset
- Nature: Binary classification
- Property: Linearly separable
- Input dimension: 2 (plus bias)
- Output dimension: 1

Since both AND and OR problems are linearly separable, they can be solved using a single-layer perceptron model.

5. Methodology

The methodology describes how the perceptron is trained using the error-correction learning rule and how error reduction and convergence behavior are analyzed.

1. Model Structure

The model used is the Perceptron, introduced by Frank Rosenblatt.

The perceptron consists of:

- Two input neurons (x_1, x_2)
- One bias input
- One output neuron
- Adjustable weights w_0, w_1, w_2

2. Mathematical Model

The net input to the neuron is calculated as:

$$y_{in} = w_0 + w_1 x_1 + w_2 x_2$$

The output is determined using a step activation function:

$$y = \begin{cases} 1, & \text{if } y_{in} \geq 0 \\ 0, & \text{if } y_{in} < 0 \end{cases}$$

3. Error-Correction Learning Rule

The perceptron updates weights using the error-correction rule:

$$w_i^{new} = w_i^{old} + \eta(T - y)x_i$$

Where:

- η= learning rate
- T= target output
- y= predicted output
- x_i= input value

This rule adjusts weights only when there is a misclassification.

4. Training Procedure

1. Initialize weights randomly (small values).
2. Select a learning rate (e.g., 0.1, 0.5, 1.0).
3. Present all training samples sequentially (one epoch).
4. Compute output using current weights.
5. Calculate error $(T-y)$.
6. Update weights if error ≠ 0.
7. Repeat for multiple epochs until:

- Total error becomes zero, or
- Maximum epochs are reached.

5. Error Measurement

At each epoch, total error is computed as:

$$E = \sum | T - y |$$

This allows plotting:

- Epoch number (x-axis)
- Total error (y-axis)

The error plot typically shows a decreasing trend for linearly separable data.

6. Learning Rate and Convergence Analysis

Different learning rates are tested to observe convergence behavior.

- Small learning rate ($\eta = 0.1$)
    - Slow convergence
    - Smooth error decrease
    - Stable updates
- Moderate learning rate ($\eta = 0.5$)
    - Faster convergence
    - Efficient learning
- Large learning rate ($\eta = 1.0$ or more)
    - Very fast updates
    - May cause oscillations before convergence

For AND and OR tasks:

- OR usually converges faster because it requires a less restrictive decision boundary.
- AND may require more updates since only one input combination produces output 1.

7. Visualization of Results

The following graphs are typically generated:

1. Error vs Epoch graph
2. Weight update progression
3. Decision boundary before and after training

The decreasing error curve demonstrates the effectiveness of error-correction learning in solving linearly separable problems.

6. Error-Correction learning on AND task

Let:

$$\eta = 1$$

Initial weights:

$$w_1 = 0, w_2 = 0, b = 0$$

Epoch 1

Input (0,0), Target 0

v = 0
y = 1 (since v ≥ 0)
Error = -1

Update:

b = 0 + (1)(-1) = -1

Input (0,1), Target 0

v = -1
y = 0
Correct → No update

Input (1,0), Target 0

v = -1

Correct → No update

Input (1,1), Target 1

v = -1

y = 0

Error = 1

Update:

$w_1 = 1$

$w_2 = 1$

b = 0

Epoch 2

Recalculate all samples.

After several updates, weights converge to:

$$w_1 = 1, w_2 = 1, b = -1.5$$

Decision boundary:

$$x_1 + x_2 - 1.5 = 0$$

All samples correctly classified → Converged.

Error Reduction Pattern (AND)

Error per epoch typically:

Epoch 1 → 2 errors

Epoch 2 → 1 error

Epoch 3 → 0 errors

Error decreases stepwise.

7. Error-Correction learning on OR task

Same initialization:

$$w_1 = 0, w_2 = 0, b = 0$$

$\eta = 1$

Epoch 1

(0,0), Target 0

Error = -1

b = -1

(0,1), Target 1

Error = 1

$w_2 = 1$

b = 0

(1,0), Target 1

Error = 1

$w_1 = 1$

b = 1

After a few updates, convergence occurs.

Final weights approximate:

$$w_1 = 1, w_2 = 1, b = -0.5$$

Decision boundary:

$$x_1 + x_2 - 0.5 = 0$$

Error Reduction Pattern (OR)

Converges faster than AND in most cases because:

- Three positive samples push weights strongly positive.

- Decision boundary is easier to establish.

8. Effect of learning rate on convergence

We test:

$\eta = 0.1$

- $\eta = 1$
- $\eta = 5$

Case 1: Small Learning Rate (η = 0.1)

- Very slow updates
- Many epochs required
- Stable
- Smooth convergence

Case 2: Moderate Learning Rate (η = 1)

- Fast convergence
- Stable
- Optimal balance

Case 3: Large Learning Rate (η = 5)

- Large jumps
- Oscillations
- May overshoot boundary
- Convergence unstable

Observation

Convergence speed increases with η but stability decreases.

Optimal learning rate ensures:

- Fast learning
- No oscillation
- Finite convergence

9. Comparison of AND vs OR learning

Feature	AND	OR
Positive samples	1	3
Convergence speed	Slower	Faster
Decision boundary	Farther from origin	Closer to origin
Weight updates	More corrective updates	Strong positive updates

Key Insight

OR learning pushes weights positively quickly.
AND requires precise boundary placement.

10. Advantages

- **Guaranteed Convergence for Linearly Separable Problems**

  The Perceptron learning rule guarantees convergence when training data (such as AND and OR gates) are linearly separable.

- **Simple and Easy to Implement**

  The simplicity makes it suitable for manual calculation, spreadsheet implementation, or beginner-level programming**.**

- **Clear Visualization of Learning Dynamics**

  By plotting error decrease over iterations, students can clearly observe how learning progresses and how the decision boundary adjusts over time.

- **Demonstrates Learning Rate Impact**

  The demo effectively shows how different learning rates (η) influence convergence speed:
  - Small η → Slow but stable convergence
  - Large η → Faster but may oscillate

- **Foundation for Advanced Algorithms**

  Error-correction learning forms the basis for more advanced techniques like gradient descent and backpropagation used in multilayer neural networks.

11.Limitations

- **Cannot Solve Non-Linearly Separable Problems**
  The perceptron fails on problems like XOR because they are not linearly separable.
- **Sensitive to Learning Rate Selection**
  If the learning rate is too high, the model may oscillate. If too low, convergence becomes very slow.
- **Binary Classification Only**
  Basic perceptron handles only two classes. It cannot directly manage multi-class problems without modification.

- **Hard Threshold Activation**
  The step activation function is non-differentiable, limiting the model's ability to use gradient-based optimization methods effectively.
- **Limited Model Complexity**
  A single-layer perceptron cannot capture complex patterns or nonlinear relationships found in real-world datasets.

12. Applications

- **Logic Gate Implementation (AND/OR Classification)**

The perceptron is ideal for modeling simple logical operations like AND and OR, which are linearly separable.

- **Basic Pattern Classification**

It can classify simple binary patterns such as spam vs non-spam (in very simplified cases).

- **Educational Demonstrations**

Widely used in academic environments to demonstrate supervised learning, convergence behavior, and error-correction principles.

- **Linear Decision Boundary Problems**

Applied in problems where data can be separated using a straight line (2D) or hyperplane (higher dimensions).

- **Foundation for Deep Learning Understanding**

Understanding perceptron error correction helps in grasping multilayer networks and backpropagation used in modern AI systems.

13. Conclusion

The Error-Correction Learning demonstration using a Perceptron on the AND and OR logic tasks clearly illustrates the effectiveness and limitations of supervised linear classifiers. By training the Perceptron using the Perceptron learning rule, the network successfully adjusted its weights to minimize classification errors for both tasks, confirming that linearly separable problems can be solved efficiently using error-driven learning.

For both the AND and OR datasets, the error decreased progressively across epochs. Initially, misclassifications were frequent due to random weight initialization. However, as the learning rule updated the weights in the direction of reducing output-target differences, the total error reduced step-by-step until convergence was achieved. The error plots demonstrated a monotonic or near-monotonic decrease, depending on the learning rate, confirming the theoretical guarantee that the Perceptron converges for linearly separable data.

The effect of the learning rate (η) played a crucial role in convergence behavior:

- Small learning rate:
  Convergence was stable but slower. The error decreased gradually, requiring more epochs to reach zero. This ensured smooth updates and minimal oscillations but increased training time.
- Moderate learning rate:
  Provided the best balance between speed and stability. The network converged quickly with minimal fluctuations in the error curve.
- Large learning rate:
  Caused instability in weight updates. The error plot showed oscillations, and in some cases convergence was delayed or inconsistent due to overshooting the optimal decision boundary.

The AND task required slightly more careful adjustment of weights compared to the OR task, as the AND function has fewer positive outputs, making correct separation more sensitive to weight updates. Nevertheless, both tasks confirmed that error-correction learning systematically modifies the decision boundary until all patterns are correctly classified.

This experiment reinforces several important theoretical insights:

1. The Perceptron converges only for linearly separable problems.
2. Error-correction learning ensures gradual minimization of classification error.
3. The learning rate directly influences convergence speed and stability.
4. Proper parameter selection is essential for efficient training.

Overall, the demonstration validates the foundational principles of supervised learning and highlights how simple neural models can effectively solve binary classification problems. Although limited to linear separability, the Perceptron and its error-correction mechanism form the conceptual basis for more advanced multilayer neural networks and backpropagation algorithms used in modern Artificial Neural Networks.